

Escuela Superior Politécnica Del Litoral
Faculty of Electrical and Computer Engineering
Software Engineering II



Acceptance Test Workshop

Group 5:

Isabella Martín – iimartin@espol.edu.ec

José Luis Chong – jlchong@espol.edu.ec

Zahid Díaz – zdiaz@espol.edu.ec

Annabella Sánchez – ansaguzm@espol.edu.ec

Juan Munizaga – juanmuni@espol.edu.ec

Gabriela Jiménez - gabdejim@espol.edu.ec

Date:

Monday, July 6th, 2026

I PAO 2026

Table of contents

1. Introduction.....	3
2. Development.....	3
2.1 <i>Environment Setup</i>	<i>3</i>
Repository Creation and Project Structure	4
Virtual Environment Configuration	4
Behave Installation.....	4
2.2 <i>Inventory Manager Design.....</i>	<i>5</i>
Product Model	5
Core Functionalities.....	5
Our group feature - Search by category.....	9
Error Handling	9
Verification	9
2.3 <i>Feature File Creation (Gherkin scenarios)</i>	<i>10</i>
2.4 <i>Acceptance Tests Implementation (steps.py).....</i>	<i>13</i>
Scenario 1: Add a product to the inventory	13
Scenario 2: List all products in the inventory	15
Scenario 3: Update the quantity of a product.....	16
2.5 <i>Test Execution and Evidence (behave run screenshots)</i>	<i>19</i>
2.6 <i>Iterative Process (Part 3: errors found and corrected)</i>	<i>21</i>
3. Conclusions.....	23
4. Recommendations.....	24
5. References.....	24

1. Introduction

Acceptance testing is the stage of software development where a system is validated against user needs and business requirements, determining whether it is ready for delivery. Behavior Driven Development (BDD) supports this process by describing expected system behavior in a shared and natural language format that both technical and nontechnical stakeholders can understand.

This workshop focuses on applying BDD principles using the Gherkin language and the Behave framework in Python. Through a simple Inventory Manager application, the goal is to design, implement and validate a set of acceptance criteria expressed as feature files and executable test scenarios. This includes writing Given-When-Then scenarios, connecting them to step definitions in Python, running the test suite and iterating on the implementation based on test results.

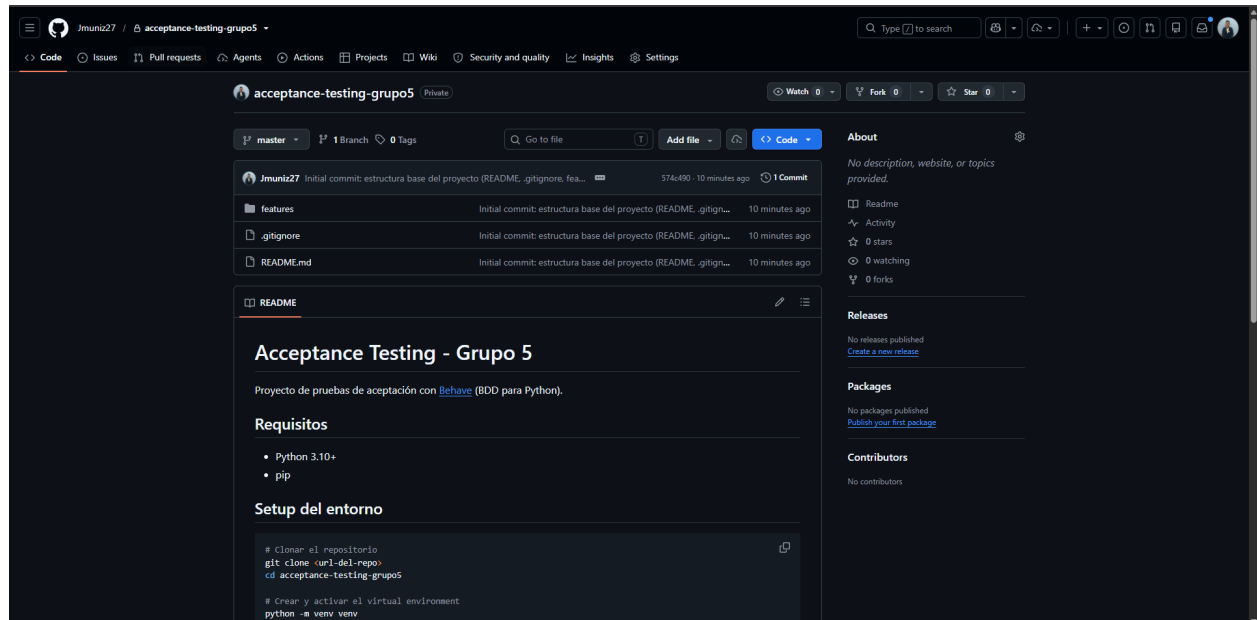
The following sections detail the environment setup, the design decisions for the Inventory Manager, the scenarios developed for each functionality, the execution results and the corrections made during the iterative testing process.

2. Development

2.1 Environment Setup

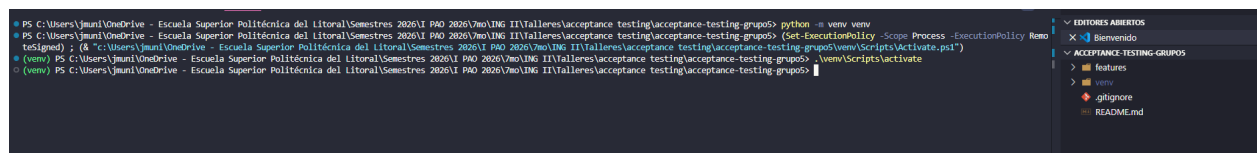
Repository Creation and Project Structure

A GitHub repository was created to host the Inventory Manager project, establishing the base folder structure required by the workshop.



Virtual Environment Configuration

A Python virtual environment was created and activated to isolate the project dependencies from the operating system. Correct activation is confirmed by the (venv) prefix appearing in the terminal.



Behave Installation

With the virtual environment active, the Behave library was installed using pip. This library enables the execution of acceptance tests written in Gherkin language.

```

(venv) PS C:\Users\jmunil\OneDrive - Escuela Superior Politécnica del Litoral\Semestres 2026\I PAO 2026\7mo\IMG II\Talleres\acceptance testing\acceptance-testing-grupo5> pip install behave
WARNING: Cache entry deserialization failed, entry ignored
Collecting behave
  Using cached behave-1.3.3-py2.py3-none-any.whl.metadata (10 kB)
WARNING: Cache entry deserialization failed, entry ignored
Collecting cucumber-tag-expressions>=4.1.0 (from behave)
  Using cached cucumber_tag_expressions-10.0.0-py3-none-any.whl.metadata (4.7 kB)
WARNING: Cache entry deserialization failed, entry ignored
Collecting cucumber-expressions>=17.1.0 (from behave)
  Using cached cucumber_expressions-20.0.0-py3-none-any.whl.metadata (1.7 kB)
WARNING: Cache entry deserialization failed, entry ignored
Collecting parse>=1.18.0 (from behave)
  Using cached parse-1.22.1-py2.py3-none-any.whl.metadata (21 kB)
WARNING: Cache entry deserialization failed, entry ignored
Collecting parse-type>=0.6.0 (from behave)
  Using cached parse_type-0.6.6-py2.py3-none-any.whl.metadata (12 kB)
WARNING: Cache entry deserialization failed, entry ignored
Collecting six>=1.15.0 (from behave)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
WARNING: Cache entry deserialization failed, entry ignored
Collecting colorama>=0.3.7 (from behave)
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Using cached behave-1.3.3-py2.py3-none-any.whl (223 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Using cached cucumber_expressions-20.0.0-py3-none-any.whl (20 kB)
Using cached cucumber_tag_expressions-10.0.0-py3-none-any.whl (9.7 kB)
Using cached parse-1.22.1-py2.py3-none-any.whl (20 kB)
Using cached parse_type-0.6.6-py2.py3-none-any.whl (27 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: parse, six, cucumber-tag-expressions, cucumber-expressions, colorama, parse-type, behave
Successfully installed behave-1.3.3 colorama-0.4.6 cucumber-expressions-20.0.0 cucumber-tag-expressions-10.0.0 parse-1.22.1 parse-type-0.6.6 six-1.17.0

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS C:\Users\jmunil\OneDrive - Escuela Superior Politécnica del Litoral\Semestres 2026\I PAO 2026\7mo\IMG II\Talleres\acceptance testing\acceptance-testing-grupo5>

```

2.2 Inventory Manager Design

Product Model

Each product in the inventory is represented as a dictionary with four attributes:

- name: a string that also is a unique identifier of the product.
- quantity: an integer that is the current stock available.
- category: string that is the classification used to group related products.
- price: a float that is the unit price of the product.

Core Functionalities

Four core functions were implemented in inventory.py to manage the product list:

- `add_product()` adds a new product to the inventory. It raises a `ProductAlreadyExistsError` if a product with the same name is already registered, preventing duplicate entries.

```
python inventory.py

Inventory Manager
1. Add product
2. List products
3. Update quantity
4. Remove product
5. Search by category
0. Exit

Choose an option: 1
Name: Apple
Quantity: 6
Category: Fruit
Price: 0.5
Product "Apple" added.
```

Duplicate entries error:

```
Inventory Manager
1. Add product
2. List products
3. Update quantity
4. Remove product
5. Search by category
0. Exit

Choose an option: 1
Name: Apple
Quantity: 40
Category: Fruit
Price: 0.4
Error: Product "Apple" already exists
```

- `list_products()` returns the full list of products currently stored in the inventory.

```
Inventory Manager
1. Add product
2. List products
3. Update quantity
4. Remove product
5. Search by category
0. Exit

Choose an option: 2
Products:
- Apple
- Black shoes
```

- `update_quantity()` updates the stock quantity of an existing product. It raises a `ProductNotFoundError` if the product does not exist.

With error:

```
Inventory Manager
1. Add product
2. List products
3. Update quantity
4. Remove product
5. Search by category
0. Exit

Choose an option: 3
Name: Rice
New quantity: 30
Error: Product "Rice" not found
```

Updated correctly:

```
Inventory Manager
1. Add product
2. List products
3. Update quantity
4. Remove product
5. Search by category
0. Exit

Choose an option: 3
Name: Apple
New quantity: 10
Product "Apple" updated to quantity 10.
```

- `remove_product()` deletes a product from the inventory. It also raises a `ProductNotFoundError` when the target product is not found.

With error:

```
Inventory Manager
1. Add product
2. List products
3. Update quantity
4. Remove product
5. Search by category
0. Exit

Choose an option: 4
Name: Phone
Error: Product "Phone" not found
```

Removed correctly:

```
Inventory Manager
1. Add product
2. List products
3. Update quantity
4. Remove product
5. Search by category
0. Exit

Choose an option: 4
Name: Black shoes
Product "Black shoes" removed.
```


Our group feature - Search by category

As the additional functionality required by the workshop `search_by_category()` was implemented. It allows the user to filter and retrieve all products that belong to a given category which supports inventory organization when the number of products grows.

```
Inventory Manager
1. Add product
2. List products
3. Update quantity
4. Remove product
5. Search by category
0. Exit

Choose an option: 5
Category: Fruit
Products:
- Apple
```

Error Handling

Two custom exceptions `ProductNotFoundError` and `ProductAlreadyExistsError` were defined to represent invalid operations. This design keeps the core logic decoupled from the command line interface and makes it easy to reuse the same functions in the Behave step definitions, where these exceptions can be caught to validate failed interaction scenarios.

Verification

The application was executed with `python inventory.py`, launching an interactive menu that exposes all five functionalities without errors.

```
isabella@Isabellas-MacBook-Air acceptance-testing-grupo5 % python3 -m venv venv
source venv/bin/activate
python inventory.py

Inventory Manager
1. Add product
2. List products
3. Update quantity
4. Remove product
5. Search by category
0. Exit

Choose an option: 1
Name: Apple
Quantity: 6
Category: Fruit
Price: 0.5
Product "Apple" added.
```

2.3 Feature File Creation (Gherkin scenarios)

As a starting point, the logic implemented in the inventory.py file was reviewed in order to understand the system's functionalities and the fifth feature defined by the group (searching for products by category). During this review, the four attributes that make up each product were also identified which are name, quantity, category and price. Having this information was necessary because the acceptance scenarios needed to reflect the actual behavior of the application.

Based on this, the features/inventory.feature file was created that contains five scenarios written in the Gherkin language that cover the five functionalities of the system. The first one is add a product to the inventory that verifies that when a product is added to an empty inventory it becomes registered. The second is list all products in the inventory that uses a data table (| Product |) to confirm that the system correctly lists all the products. The third is update the quantity of a product that uses a table (| Product | Quantity |) to validate the update of the quantity of an existing product. The fourth is remove a product from the inventory that confirms that a removed product no longer belongs to the inventory. Finally, the fifth scenario is search products by category that corresponds to the group's additional functionality and exercises the search by category, verifying that only the products belonging to the queried category are returned.

```

1  # language: en
2  Feature: Inventory management
3    As a store manager
4    I want to add, list, update, remove and search products in my inventory
5    So that I can keep my stock information accurate and up to date
6
7    Scenario: Add a product to the inventory
8      Given the inventory is empty
9      When the user adds a product "Coffee"
10     Then the inventory should contain "Coffee"
11
12    Scenario: List all products in the inventory
13      Given the inventory contains products:
14        | Product |
15        | Coffee  |
16        | Sugar   |
17      When the user lists all products
18      Then the output should contain:
19        """
20        Products:
21        - Coffee
22        - Sugar
23        """
24
25    Scenario: Update the quantity of a product
26      Given the inventory contains products:
27        | Product | Quantity |
28        | Coffee  | 10       |
29      When the user updates product "Coffee" to quantity "25"
30      Then the inventory should show product "Coffee" with quantity "25"
31
32    Scenario: Remove a product from the inventory
33      Given the inventory contains products:
34        | Product |
35        | Coffee  |
36        | Sugar   |
37      When the user removes the product "Coffee"
38      Then the inventory should not contain "Coffee"
39
40    Scenario: Search products by category
41      Given the inventory contains products:
42        | Product | Category |
43        | Coffee  | Beverages |
44        | Tea     | Beverages |
45        | Sugar   | Grocery   |
46      When the user searches for products in category "Beverages"
47      Then the output should contain:
48        """
49        Products:
50        - Coffee
51        - Tea
52        """

```

Each scenario was written respecting the Gherkin keywords (Feature, Scenario, Given, When, Then), proper indentation, the use of data tables and doc strings (""") to validate multi-line text outputs. To ensure that the file was correctly formed, the behave --dry-run command was executed which parses the .feature files without running the application logic. The result

confirmed that the five scenarios together with their tables and doc strings, were recognized without syntax errors. The steps appeared as undefined (15 in total), which is the expected behavior at this stage since the implementation of the steps in Python is carried out afterwards.

```
ASanchez-Fedora@ASanchez-fedora:~/taller_acceptance/acceptance-testing-grupo5$ source venv/bin/activate && behave --dry-run --no-snippets
USING RUNNER: behave.runner:Runner
Feature: Inventory management # features/inventory.feature:2
  As a store manager
  I want to add, list, update, remove and search products in my inventory
  So that I can keep my stock information accurate and up to date
  Scenario: Add a product to the inventory # features/inventory.feature:7
    Given the inventory is empty # None
    When the user adds a product "Coffee" # None
    Then the inventory should contain "Coffee" # None

  Scenario: List all products in the inventory # features/inventory.feature:12
    Given the inventory contains products: # None
      | Product |
      | Coffee |
      | Sugar |
    When the user lists all products # None
    Then the output should contain: # None
      """
      Products:
      - Coffee
      - Sugar
      """

  Scenario: Update the quantity of a product # features/inventory.feature:25
    Given the inventory contains products: # None
      | Product | Quantity |
      | Coffee | 10 |
    When the user updates product "Coffee" to quantity "25" # None
    Then the inventory should show product "Coffee" with quantity "25" # None

  Scenario: Remove a product from the inventory # features/inventory.feature:32
    Given the inventory contains products: # None
      | Product |
      | Coffee |
      | Sugar |
    When the user removes the product "Coffee" # None
    Then the inventory should not contain "Coffee" # None
```

```
Scenario: Search products by category # features/inventory.feature:40
  Given the inventory contains products: # None
    | Product | Category |
    | Coffee | Beverages |
    | Tea | Beverages |
    | Sugar | Grocery |
  When the user searches for products in category "Beverages" # None
  Then the output should contain: # None
    """
    Products:
    - Coffee
    - Tea
    """

Errored scenarios:
features/inventory.feature:7 Add a product to the inventory
features/inventory.feature:12 List all products in the inventory
features/inventory.feature:25 Update the quantity of a product
features/inventory.feature:32 Remove a product from the inventory
features/inventory.feature:40 Search products by category

0 features passed, 0 failed, 1 error, 0 skipped
0 scenarios passed, 0 failed, 5 error, 0 skipped
0 steps passed, 0 failed, 0 skipped, 15 undefined
Took 0min 0.000s
```

2.4 Acceptance Tests Implementation (steps.py)

Scenario 1: Add a product to the inventory

```
@given('the inventory is empty')
def step_impl(context):
    context.inventory = create_inventory()
```

The inventory is stored in *context.inventory* because *Behave* provides a fresh context for every scenario, ensuring test isolation. We use *create_inventory()* instead of `[]` so the step remains valid even if the inventory implementation changes.

```
@when('the user adds a product "{product}"')
def step_impl(context, product):
    add_product(context.inventory, product, quantity=1, category="General", price=0.0)
```

The feature only specifies the product name, while *add_product()* requires additional arguments. We provide default values without changing the business logic.

```
@then('the inventory should contain "{product}"')
def step_impl(context, product):
    assert find_product(context.inventory, product) is not None, \
        f'Product "{product}" not found in the inventory'
```

We reuse *find_product()* instead of implementing it from scratch, avoiding duplicated logic and testing the application's behavior.

Scenario 2: List all products in the inventory

```
@given('the inventory contains products:')
def step_impl(context):
    context.inventory = create_inventory()
    has_quantity = "Quantity" in context.table.headings
    for row in context.table:
        quantity = int(row["Quantity"]) if has_quantity else 1
        add_product(context.inventory, row["Product"], quantity=quantity,
                    category="General", price=0.0)
```

We iterate through *context.table* to populate the inventory. The same step is reused across scenarios by checking whether the optional *Quantity* column exists, defaulting to 1 otherwise

```
@when('the user lists all products')
def step_impl(context):
    context.output = format_inventory(list_products(context.inventory))
```

We combine *list_products* and *format_inventory*, storing the formatted output in *context.output* for later verification.

```
@then('the output should contain:')
def step_impl(context):
    expected = context.text.strip()
    assert expected in context.output, f'Expected:\n{expected}\nGot:\n{context.output}'
```

Multiline text from the feature file is available as *context.text*. Using *.strip()* avoids failures caused by extra whitespace.

Scenario 3: Update the quantity of a product

```
@when('the user updates product "{product}" to quantity "{quantity}"')
def step_impl(context, product, quantity):
    update_quantity(context.inventory, product, int(quantity))
```

Behave captures values as strings, so quantity is converted to int before calling *update_quantity*

```
@then('the inventory should show product "{product}" with quantity "{quantity}"')
def step_impl(context, product, quantity):
    found = find_product(context.inventory, product)
    assert found is not None, f'Product "{product}" not found in the inventory'
    assert str(found["quantity"]) == quantity, \
        f'Expected quantity "{quantity}" but got "{found["quantity"]}"'
```

We first verify that the product exists, then compare the quantity. Converting the stored value to *str* ensures a valid comparison with the library string input.

Scenario 4: Remove a product from the inventory

```
@when('the user removes the product "{product}"')
def step_impl(context, product):
    remove_product(context.inventory, product)
```

The step directly calls *remove_product()* with the inventory stored in *context*. No extra processing is needed because the feature already provides all the required information.

```
@then('the inventory should not contain "{product}"')
def step_impl(context, product):
    assert find_product(context.inventory, product) is None, \
        f'Product "{product}" should not be in the inventory'
```

Instead of manually checking every product, we reuse *find_product()*. If it returns *None*, the product was successfully removed from the inventory.

Scenario 5: Search product by category

```
@when('the user searches for products in category "{category}"')
def step_impl(context, category):
    context.output = format_inventory(search_by_category(context.inventory, category))
```

We reuse *search_by_category()* to retrieve the matching products and *format_inventory()* to generate the user-facing output. The result is stored in *context.output* so it can be verified in the following *then* step.

2.5 Test Execution and Evidence (behave run screenshots)

Once all five step definitions were implemented, the complete acceptance test suite was executed with the behave command. The framework matched every Given-When-Then step in the five scenarios against its corresponding Python step definition and ran them against the Inventory Manager logic. All scenarios passed successfully, confirming that the application behaves as specified by the acceptance criteria. The final execution reported 1 feature passed, 5 scenarios passed and 15 steps passed, with 0 failures, which validates the add, list, update, remove and search-by-category functionalities end to end.

```

zdiaz-jelou@zahidiaz-fedora:~/espol/ing/acceptance-testing-grupo5$ behave
USING RUNNER: behave.runner.Runner
Feature: Inventory management # features/inventory.feature:2
  As a store manager
  I want to add, list, update, remove and search products in my inventory
  So that I can keep my stock information accurate and up to date
  Scenario: Add a product to the inventory # features/inventory.feature:7
    Given the inventory is empty # features/steps/inventory_steps.py:16 0.001s
    When the user adds a product "Coffee" # features/steps/inventory_steps.py:21 0.000s
    Then the inventory should contain "Coffee" # features/steps/inventory_steps.py:27 0.000s

  Scenario: List all products in the inventory # features/inventory.feature:12
    Given the inventory contains products: # features/steps/inventory_steps.py:35 0.000s
      | Product |
      | Coffee |
      | Sugar |
    When the user lists all products # features/steps/inventory_steps.py:47 0.000s
    Then the output should contain: # features/steps/inventory_steps.py:52 0.000s
      """
      Products:
      - Coffee
      - Sugar
      """

  Scenario: Update the quantity of a product # features/inventory.feature:25
    Given the inventory contains products: # features/steps/inventory_steps.py:35 0.000s
      | Product | Quantity |
      | Coffee | 10       |
    When the user updates product "Coffee" to quantity "25" # features/steps/inventory_steps.py:60 0.000s
    Then the inventory should show product "Coffee" with quantity "25" # features/steps/inventory_steps.py:65 0.000s

  Scenario: Remove a product from the inventory # features/inventory.feature:32
    Given the inventory contains products: # features/steps/inventory_steps.py:35 0.000s
      | Product |
      | Coffee |
      | Sugar |
    When the user removes the product "Coffee" # features/steps/inventory_steps.py:75 0.000s
    Then the inventory should not contain "Coffee" # features/steps/inventory_steps.py:80 0.000s


  Scenario: Search products by category # features/inventory.feature:40
    Given the inventory contains products: # features/steps/inventory_steps.py:35 0.000s
      | Product | Category |
      | Coffee | Beverages |
      | Tea   | Beverages |
      | Sugar | Grocery   |
    When the user searches for products in category "Beverages" # features/steps/inventory_steps.py:88 0.000s
    Then the output should contain: # features/steps/inventory_steps.py:92 0.000s
      """
      Products:
      - Coffee
      - Tea
      """

1 feature passed, 0 failed, 0 skipped
5 scenarios passed, 0 failed, 0 skipped
15 steps passed, 0 failed, 0 skipped
Took 0min 0.003s
zdiaz-jelou@zahidiaz-fedora:~/espol/ing/acceptance-testing-grupo5$

```

2.6 Iterative Process (Part 3: errors found and corrected)

During the iterative testing process the suite was executed repeatedly, and two issues were detected and corrected before reaching a fully passing state. Each error, its root cause and the applied fix are documented below.

2.6.1. Error 1 – Behave version incompatibility

On the first execution, four of the five scenarios failed and the steps appeared as *undefined*, even though they were already implemented. The root cause was that the environment had Behave 1.2.6 installed, a version that strips the trailing colon (:) from a step when it is followed by a data table or a doc string. As a result, the steps the inventory contains products: and the output should contain: could not be matched. The issue was corrected by upgrading the framework with pip install --upgrade behave to Behave 1.3.3. After the upgrade, the same steps were matched correctly and the suite executed without errors.

```
zdzia-jelou@zahidiaz-fedora:~/espol/ing/acceptance-testing-grupo5$ behave
Feature: Inventory management # features/inventory.feature:2
  As a store manager
  I want to add, list, update, remove and search products in my inventory
  So that I can keep my stock information accurate and up to date
  Scenario: Add a product to the inventory # features/inventory.feature:7
    Given the inventory is empty # features/steps/inventory_steps.py:16 0.000s
    When the user adds a product "Coffee" # features/steps/inventory_steps.py:21 0.000s
    Then the inventory should contain "Coffee" # features/steps/inventory_steps.py:27 0.000s

  Scenario: List all products in the inventory # features/inventory.feature:12
    Given the inventory contains products # None
      | Product |
      | Coffee |
      | Sugar |
    When the user lists all products # None
    Then the output should contain # None
      """
      Products:
      - Coffee
      - Sugar
      """

  Scenario: Update the quantity of a product # features/inventory.feature:25
    Given the inventory contains products # None
      | Product | Quantity |
      | Coffee | 10      |
    When the user updates product "Coffee" to quantity "25" # None
    Then the inventory should show product "Coffee" with quantity "25" # None

  Scenario: Remove a product from the inventory # features/inventory.feature:32
    Given the inventory contains products # None
      | Product |
      | Coffee |
      | Sugar |
    When the user removes the product "Coffee" # None
    Then the inventory should not contain "Coffee" # None
```

```

Scenario: Search products by category                                # features/inventory.feature:40
  Given the inventory contains products                             # None
    | Product | Category |
    | Coffee  | Beverages |
    | Tea     | Beverages |
    | Sugar   | Grocery   |
  When the user searches for products in category "Beverages" # None
  Then the output should contain                                  # None
    """
    Products:
    - Coffee
    - Tea
    """

Failing scenarios:
  features/inventory.feature:12 List all products in the inventory
  features/inventory.feature:25 Update the quantity of a product
  features/inventory.feature:32 Remove a product from the inventory
  features/inventory.feature:40 Search products by category

0 features passed, 1 failed, 0 skipped
1 scenario passed, 4 failed, 0 skipped
3 steps passed, 0 failed, 6 skipped, 6 undefined
Took 0m0.000s

You can implement step definitions for undefined steps with these snippets:

@given(u'the inventory contains products')
def step_impl(context):
    raise NotImplementedError(u'STEP: Given the inventory contains products')

@then(u'the output should contain')
def step_impl(context):
    raise NotImplementedError(u'STEP: Then the output should contain')

zdiaz-jelou@zahidiaz-fedora:~/espol/ing/acceptance-testing-grupo5$

```

2.6.2. Error 2 – Category column not read from the data table

A second error appeared in the *Search products by category* scenario. The shared step Given the inventory contains products: only read the Product and Quantity columns and assigned a fixed category "General" to every product. Because of this, searching for the "Beverages" category returned an empty result and the assertion failed (expected Coffee and Tea, but the output contained no products). The step was corrected to detect the Category column dynamically from the table headings, applying the same pattern already used for Quantity, so each product keeps its real category. After this correction the scenario returned the expected products and the full suite passed with 5 scenarios and 0 failures.

```

zdiaz-jelou@zahidiaz-fedora:~/espol/ing/acceptance-testing-grupo5$ behave
USING RUNNER: behave.runner:Runner
Feature: Inventory management # features/inventory.feature:2
  As a store manager
  I want to add, list, update, remove and search products in my inventory
  So that I can keep my stock information accurate and up to date
  Scenario: Add a product to the inventory # features/inventory.feature:7
    Given the inventory is empty # features/steps/inventory_steps.py:16 0.000s
    When the user adds a product "Coffee" # features/steps/inventory_steps.py:21 0.000s
    Then the inventory should contain "Coffee" # features/steps/inventory_steps.py:27 0.000s

  Scenario: List all products in the inventory # features/inventory.feature:12
    Given the inventory contains products: # features/steps/inventory_steps.py:35 0.000s
      | Product |
      | Coffee |
      | Sugar |
    When the user lists all products # features/steps/inventory_steps.py:47 0.000s
    Then the output should contain: # features/steps/inventory_steps.py:52 0.000s
      """
      Products:
      - Coffee
      - Sugar
      """

  Scenario: Update the quantity of a product # features/inventory.feature:25
    Given the inventory contains products: # features/steps/inventory_steps.py:35 0.000s
      | Product | Quantity |
      | Coffee | 10       |
    When the user updates product "Coffee" to quantity "25" # features/steps/inventory_steps.py:60 0.000s
    Then the inventory should show product "Coffee" with quantity "25" # features/steps/inventory_steps.py:65 0.000s

  Scenario: Remove a product from the inventory # features/inventory.feature:32
    Given the inventory contains products: # features/steps/inventory_steps.py:35 0.000s
      | Product |
      | Coffee |
      | Sugar |
    When the user removes the product "Coffee" # features/steps/inventory_steps.py:75 0.000s
    Then the inventory should not contain "Coffee" # features/steps/inventory_steps.py:80 0.000s

```

3. Conclusions

The workshop allowed the team to apply Behavior Driven Development principles in a practical environment, bridging the gap between business requirements and technical implementation through the use of Gherkin language and the Behave framework. Acceptance testing was validated as an effective method to verify that a system meets user needs and business processes before delivery. The Inventory Manager project demonstrated that BDD scenarios written with Given, When, and Then steps produce tests that are readable by both technical and non-technical team members, making requirements easier to understand and validate. The use of the context object in Behave proved to be a cleaner and more reliable approach than global variables for sharing state between steps, as it prevents data leakage between scenarios. The iterative process in Part 3 reinforced the importance of running tests continuously during development, as it allows errors to be detected and corrected early rather than at the end of the project.

4. Recommendations

- Define acceptance scenarios before implementing new functionalities so that requirements are clearly understood by both developers and stakeholders.
- Keep Gherkin scenarios simple, readable, and focused on business behavior instead of implementation details.
- Continue using custom exceptions and reusable helper functions to improve code maintainability and reduce duplication.
- Execute the Behave test suite after every significant code change to detect regressions as early as possible.
- Expand the acceptance test suite by including additional negative scenarios, boundary conditions, and invalid user inputs to increase software reliability.
- Integrate acceptance tests into a Continuous Integration (CI) pipeline so they are executed automatically whenever changes are pushed to the repository.

5. References

1. Behave. (n.d.). Behave documentation. <https://behave.readthedocs.io/en/latest/>
2. Cucumber. (n.d.). Gherkin Reference. <https://cucumber.io/docs/gherkin/reference/>
3. Hewitt, R. (2017). Gherkin for Business Analysts. Modern Analyst.
<https://www.modernanalyst.com/Resources/Articles/tabid/115/ID/3810/Gherkin-for-Business-Analysts.aspx>
4. Software Testing Fundamentals. (n.d.). Acceptance Testing.
<http://softwaretestingfundamentals.com/acceptance-testing/>
5. Vergara, S. (2019, July 18). ¿Qué es BDD (Behavior Driven Development)? ITDO.
<https://www.itdo.com/blog/que-es-bdd-behavior-driven-development/>